Stage 1 — Core LLM + Tool Use: The Single Agent Loop

> **Goal:** Understand how a single AI agent works from the inside out — messages, tool calling, the agent loop — before touching any framework. Build a real working agent from scratch using Python and the OpenAI API.

Why Start Here (and Not with LangChain)

Before reaching for LangChain, LangGraph, or CrewAI, you need to understand what they are *wrapping*. Every agent framework, no matter how complex, is ultimately built on three primitives:

1. **Messages** — structured conversations between you and the model.
2. **Tool calling** — the model outputs a structured JSON "request" to call a function; your code executes it and feeds the result back.
3. **The agent loop** — repeat (call model → check for tool call → execute tool → feed result back) until the model gives a final text answer.

If you learn a framework first, you'll use it without understanding *why* it works, and you'll be stuck when it breaks or when you need to customize it.

The recommendation for Stage 1: use the raw OpenAI SDK in Python. No LangChain.

LangChain is excellent once you understand the primitives — it adds multi-provider abstraction, retrieval chains, and integration components. But for Stage 1, it adds abstraction before you've earned it. Once you finish Stage 1, LangChain will feel like a shortcut you understand, not a magic box you depend on.

Framework Decision Table

Option	Best for	Stage 1 verdict
Raw OpenAI SDK	Learning fundamentals, full control	✅ Use this
LangChain	Multi-provider apps, RAG, rapid chaining	❌ Stage 2–3
LangGraph	Stateful multi-agent graphs	❌ Stage 2–3
CrewAI	Role-based multi-agent prototyping	❌ Stage 3+
OpenAI Agents SDK	Production OpenAI-native agents	❌ Stage 2+

Concept 1 — Messages and Roles

Every LLM conversation is just a list of messages. Each message has a `role` and `content`.

Role	Who	Purpose
`system`	You (developer)	Defines the agent's identity, behavior, rules
`user`	The human	Input / query
`assistant`	The model	The model's response (text or tool call)
`tool`	Your code	The result of running a tool the model requested

The model sees the **entire list every time** you call it. This is the context window. Order matters. History matters.

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user",   "content": "What is 25% of 480?"},
]
```

Key Mental Model

Think of messages as a **running transcript of a conversation** between:

- The developer (system) who sets the rules once.
- The user who speaks.
- The assistant who responds.
- Your code (tool) that executes things on behalf of the assistant.

Concept 2 — Tool Calling (Function Calling)

Tool calling is the mechanism that turns a chatbot into an **agent**.

Without tool calling: model generates text → done.
With tool calling: model can say "please run this function with these arguments" → you run it → you feed the result back → model gives a final answer.

The model does NOT execute tools. It outputs a structured JSON saying "I want to call `tool_name` with these `arguments`." Your code does the actual execution. This is a critical security and architectural boundary.

The 5-Step Tool Calling Loop

```
Step 1: You define tools as JSON schemas
        (name, description, parameters)

Step 2: You call the model with: messages + tool definitions
```

```
Step 3: Model responds with a tool_call
        { "name": "calculator", "arguments": {"expression": "480 * 0.25"} }

Step 4: Your code executes: calculator("480 * 0.25") → "120.0"

Step 5: You add the result to messages as a "tool" role message
        and call the model again → model gives final answer
```

This loop can repeat multiple times if the model needs several tool calls to answer the question.

Concept 3 — The Agent Loop

The **agent loop** is the `while` loop that keeps running until the model produces a final text answer (no more tool calls).

```
while True:
    response = call_model(messages)

    if response has tool_calls:
        for each tool_call:
            result = execute_tool(tool_call.name, tool_call.arguments)
            append tool result to messages
        # loop again — feed results back to model

    else:
        # model gave a text answer — we're done
        return response.text
```

This is the core of every AI agent. LangChain, CrewAI, LangGraph — all of them implement some version of this loop. You are now going to implement it yourself.

Project Setup

Prerequisites

- Python 3.10+
- An OpenAI API key (get one at platform.openai.com)
- Basic Python knowledge (you already have this)

Folder Structure

```
stage1-agent/
├── .env                  ← API keys (never commit this)
├── .env.example          ← Template for .env
├── requirements.txt
├── config.py             ← Loads config from .env
├── tools/
```

```
│   ├── __init__.py
│   ├── registry.py       ← Central tool registry
│   ├── calculator.py     ← Tool 1
│   ├── weather.py        ← Tool 2 (simulated)
│   └── word_count.py     ← Tool 3
├── agent/
│   ├── __init__.py
│   ├── loop.py           ← The agent loop
│   └── prompts.py        ← System prompts
└── main.py               ← Entry point (CLI)
```

This is your **first taste of enterprise-style separation**:

- `tools/` is your early tool registry.
- `agent/` is your orchestration logic.
- `config.py` keeps secrets out of code.

Step 1 — Environment Setup

requirements.txt

```
openai>=1.30.0
python-dotenv>=1.0.0
```

Install:

```
pip install -r requirements.txt
```

.env.example

```
OPENAI_API_KEY=your_key_here
MODEL_NAME=gpt-4o-mini
```

Create your actual `.env` by copying this and filling in your key:

```
cp .env.example .env
```

config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

```
MODEL_NAME     = os.getenv("MODEL_NAME", "gpt-4o-mini")

if not OPENAI_API_KEY:
    raise ValueError("OPENAI_API_KEY is not set in .env")
```

Why this matters: This pattern — load from `.env`, validate at startup, expose typed config — is exactly what the enterprise multi-LLM layer will use later. You're building the habit now.

Step 2 — Define Your Tools

Each tool has two parts:

1. **The JSON schema** — tells the model what the tool does and what arguments it takes.
2. **The Python function** — the actual implementation your code runs.

tools/calculator.py

```
import ast
import operator

# --- JSON Schema (sent to the model) ---
SCHEMA = {
    "type": "function",
    "function": {
        "name": "calculator",
        "description": (
            "Evaluates a safe mathematical expression. "
            "Use this for any arithmetic, percentages, or calculations."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "expression": {
                    "type": "string",
                    "description": "A valid math expression, e.g. '480 * 0.25' or '(
                }
            },
            "required": ["expression"]
        }
    }
}

# --- Implementation (run by your code, NOT the model) ---
_SAFE_OPS = {
    ast.Add: operator.add,
    ast.Sub: operator.sub,
    ast.Mult: operator.mul,
    ast.Div: operator.truediv,
    ast.Pow: operator.pow,
    ast.USub: operator.neg,
}
```

```
def _safe_eval(node):
    if isinstance(node, ast.Constant):
        return node.value
    elif isinstance(node, ast.BinOp):
        return _SAFE_OPS[type(node.op)](
            _safe_eval(node.left), _safe_eval(node.right)
        )
    elif isinstance(node, ast.UnaryOp):
        return _SAFE_OPS[type(node.op)](_safe_eval(node.operand))
    raise ValueError(f"Unsafe expression: {ast.dump(node)}")

def run(expression: str) -> str:
    """Execute the calculator tool safely."""
    try:
        tree = ast.parse(expression, mode="eval")
        result = _safe_eval(tree.body)
        return str(round(result, 6))
    except Exception as e:
        return f"Error: {e}"
```

Why safe eval? Because tools that execute arbitrary code are dangerous. Enterprise guardrails start here — the sandboxing concept you'll expand in Stage 4 starts with safe, limited tool implementations.

tools/weather.py

```
# Simulated weather tool — no real API needed for learning
SCHEMA = {
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Gets the current weather for a given city. Returns temperatu
        "parameters": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",
                    "description": "The city name, e.g. 'Toronto' or 'New York'"
                }
            },
            "required": ["city"]
        }
    }
}

# Simulated data — in production, this would call a real weather API
_FAKE_WEATHER = {
    "toronto":   {"temp_c": 22, "condition": "Partly cloudy"},
    "new york":  {"temp_c": 28, "condition": "Sunny"},
    "london":    {"temp_c": 15, "condition": "Overcast"},
    "tokyo":     {"temp_c": 30, "condition": "Humid and warm"},
}
```

```
def run(city: str) -> str:
    """Execute the weather tool."""
    data = _FAKE_WEATHER.get(city.lower())
    if not data:
        return f"No weather data available for '{city}'."
    return (
        f"{city.title()}: {data['temp_c']}°C, {data['condition']}"
    )
```

tools/word_count.py

```
SCHEMA = {
    "type": "function",
    "function": {
        "name": "word_count",
        "description": "Counts the number of words and characters in a given text.",
        "parameters": {
            "type": "object",
            "properties": {
                "text": {
                    "type": "string",
                    "description": "The text to analyze"
                }
            },
            "required": ["text"]
        }
    }
}

def run(text: str) -> str:
    """Execute the word count tool."""
    words = len(text.split())
    chars = len(text)
    chars_no_space = len(text.replace(" ", ""))
    return (
        f"Words: {words} | "
        f"Characters (with spaces): {chars} | "
        f"Characters (no spaces): {chars_no_space}"
    )
```

tools/registry.py

This is your **Tool Registry** — the central place that maps tool names to their schemas and implementations. This is a small version of what your enterprise CreatorOrchestrator will have.

```
from tools import calculator, weather, word_count

# All registered tools: maps tool name → (schema, run_function)
REGISTRY = {
```

```
    "calculator": (calculator.SCHEMA, calculator.run),
    "get_weather": (weather.SCHEMA,   weather.run),
    "word_count":  (word_count.SCHEMA, word_count.run),
}

def get_all_schemas() -> list:
    """Return all tool schemas — passed to the model so it knows what's available."""
    return [schema for schema, _ in REGISTRY.values()]

def execute(tool_name: str, arguments: dict) -> str:
    """Execute a tool by name with the given arguments."""
    if tool_name not in REGISTRY:
        return f"Error: unknown tool '{tool_name}'"
    _, run_fn = REGISTRY[tool_name]
    try:
        return run_fn(**arguments)
    except Exception as e:
        return f"Tool execution error: {e}"
```

Key insight: The registry decouples the *definition* of a tool from the *execution*. The model only ever sees the schemas (JSON). It never sees or touches the `run()` functions. Your orchestrator later can filter which tools are available to which agent — just change what `get_all_schemas()` returns.

Step 3 — System Prompts

agent/prompts.py

```
SYSTEM_PROMPT = """You are a helpful assistant with access to tools.

When answering:
- Use the calculator tool for any math or percentage calculations.
- Use get_weather for any weather questions.
- Use word_count to analyze text length.
- If a task requires multiple steps, do them one at a time.
- Always explain your reasoning briefly before giving the final answer.
- If you cannot answer with available tools, say so clearly.
"""
```

Why a separate file? Your system prompt IS your agent's behavior. Later, each specialist agent (IngestionAgent, CopyAgent, BrandGuardAgent) will have its own prompt file. The habit of keeping prompts separate from logic is crucial for maintainability.

Step 4 — The Agent Loop

This is the most important file. Read it carefully.

agent/loop.py

```
import json
from openai import OpenAI
from config import OPENAI_API_KEY, MODEL_NAME
from tools.registry import get_all_schemas, execute
from agent.prompts import SYSTEM_PROMPT

client = OpenAI(api_key=OPENAI_API_KEY)

MAX_ITERATIONS = 10  # Safety limit — prevents infinite loops


def run_agent(user_message: str) -> str:
    """
    Run the single-agent loop.

    1. Build message list (system + user).
    2. Call model with tool schemas.
    3. If model returns tool_calls → execute each → add results → loop.
    4. If model returns text → return it.
    """
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user",   "content": user_message},
    ]
    tools = get_all_schemas()

    for iteration in range(MAX_ITERATIONS):
        print(f"\n[Loop iteration {iteration + 1}]")

        # ── Call the model ──────────────────────────────────────
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=messages,
            tools=tools,
            tool_choice="auto",  # model decides whether to call a tool
        )

        message = response.choices[0].message

        # ── Check: did the model want to call tools? ────────────
        if message.tool_calls:
            print(f"  Model requested {len(message.tool_calls)} tool call(s):")

            # Add the assistant's tool-call message to history
            messages.append(message)

            # Execute each tool and add results
            for tool_call in message.tool_calls:
                tool_name = tool_call.function.name
                arguments = json.loads(tool_call.function.arguments)

                print(f"    → Calling '{tool_name}' with {arguments}")
                result = execute(tool_name, arguments)
                print(f"    ← Result: {result}")
```

```
                # Add tool result to messages (role="tool")
                messages.append({
                    "role":         "tool",
                    "tool_call_id": tool_call.id,
                    "content":      result,
                })

            # Continue the loop — feed results back to model

        else:
            # ── Model gave a final text answer ──────────────────────────
            final_answer = message.content
            print(f"\n[Final answer after {iteration + 1} iteration(s)]")
            return final_answer

    return "Error: agent loop exceeded maximum iterations."
```

What to observe in this code

- `messages.append(message)` — you always add the assistant's tool-call response to history. The model needs to see its own previous tool requests.
- `tool_call_id` — every tool result is linked back to a specific tool call ID. This is how the model matches results to requests.
- `MAX_ITERATIONS` — the safety guardrail. Without this, a buggy tool or looping model could run forever and cost money. This is your first taste of guardrail design.
- `tool_choice="auto"` — the model decides whether to use tools. You can also set `"none"` (no tools) or `{"type": "function", "function": {"name": "calculator"}}` to force a specific tool.

Step 5 — Entry Point

main.py

```
from agent.loop import run_agent


def main():
    print("=== Stage 1 Agent — Single Agent + Tool Use ===")
    print("Type 'exit' to quit.\n")

    while True:
        user_input = input("You: ").strip()
        if not user_input:
            continue
        if user_input.lower() == "exit":
            print("Goodbye.")
            break
```

```
        answer = run_agent(user_input)
        print(f"\nAgent: {answer}\n")
        print("-" * 50)


if __name__ == "__main__":
    main()
```

Run It

```
python main.py
```

Test queries to try (in order of complexity)

Test 1 — Single tool call:

```
You: What is 25% of 480?
```

Expected: Model calls `calculator("480 * 0.25")` → returns "120.0" → gives answer.

Test 2 — Multi-step (two tool calls in one loop):

```
You: What is the weather in Toronto, and what is 15% of the temperature in Celsius?
```

Expected: Model calls `get_weather("Toronto")` → gets 22°C → calls `calculator("22 * 0.15")` → chains results.

Test 3 — No tool needed:

```
You: What is the capital of Canada?
```

Expected: Model answers directly without any tool call.

Test 4 — Text analysis:

```
You: Count the words in this sentence: "The quick brown fox jumps over the lazy dog"
```

Expected: Model calls `word_count(...)` → returns stats.

Test 5 — Complex chained reasoning:

```
You: If it costs $1,200 per month to rent an apartment in Toronto and rent goes up 3
```

Expected: Model chains multiple calculator calls to compute compound increases.

What You Should Understand After This

After running the agent and looking at the loop output for each test:

Q&A to test yourself

Q: Why does the loop sometimes run 2 or 3 iterations for one question?
A: Because the model may call a tool, get the result, then call another tool, get that result, and only then give a final answer. The loop feeds results back each time.

Q: What happens if you remove a tool from the registry?
A: The model simply won't know it exists (it's not in the schemas anymore). Test this — remove `word_count` from the registry and ask a word-count question.

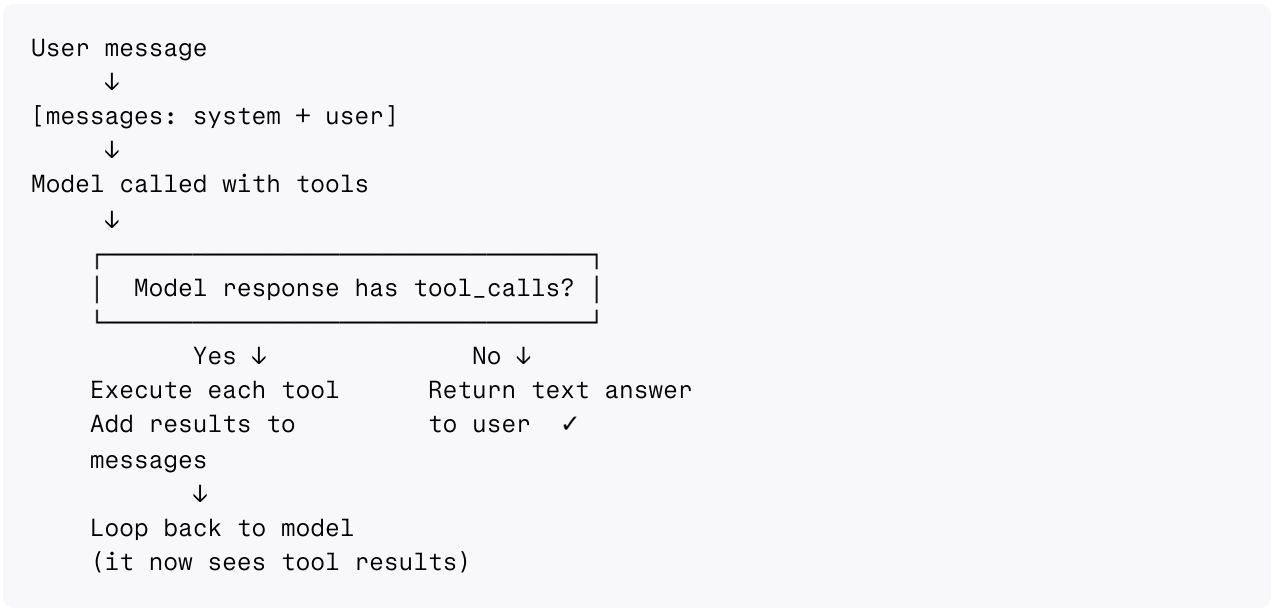
Q: What is the difference between `role: assistant` (with tool_calls) and `role: assistant` (with just `content`)?
A: `tool_calls` present = model wants to act. `content` present and no `tool_calls` = model is done and giving you the answer.

Q: What would happen without `MAX_ITERATIONS`?
A: A buggy tool that always returns an error, or a model that keeps requesting the same tool in a loop, would run forever and consume API credits. This is why every production agent loop has a safeguard.

Q: If you wanted to add a new tool (e.g., `web_search`), what files do you touch?
A: Create `tools/web_search.py` with a `SCHEMA` and a `run()` function. Register it in `tools/registry.py`. Zero changes to the agent loop. This is the **open/closed principle** — open to extension, closed to modification.

Stage 1 — Mental Model Checkpoint

At this point you should be able to visualize this when any agent runs:



This loop is the **heartbeat of every AI agent** in existence. LangGraph wraps it in a state graph. CrewAI wraps it with role-based coordination. LangChain wraps it in a chain. But underneath, it's always this.

What to Build Next (Mini-Project 1 Completion Criteria)

You've completed Stage 1 when you can:

- [ ] Run the CLI agent and see it call tools correctly.
- [ ] Add a new tool (`current_date`) without touching `agent/loop.py`.
- [ ] Explain out loud what happens to `messages` when a tool is called.
- [ ] Understand why `MAX_ITERATIONS` matters.
- [ ] Ask a multi-step question and trace every loop iteration in the console output.

Stretch goal: Add a `summarize_text` tool that accepts text and returns a bullet-point summary by calling Claude (Anthropic API) instead of OpenAI — this is your first taste of multi-LLM routing. Use the `anthropic` Python SDK (`pip install anthropic`).

What Changes in Stage 2

Once you're solid on Stage 1, Stage 2 builds on this exact loop in two ways:

1. **Orchestrator pattern:** Instead of one agent, a "manager" agent has *other agents* as its tools. When it "calls" a ResearchAgent, it's just calling a function that runs another agent loop internally.
2. **Structured outputs:** Instead of free-text answers from sub-agents, you'll use JSON-schema responses so the orchestrator can reliably parse and route results.

Everything you built in Stage 1 — the registry, the loop, the prompts file, the `.env` config — carries forward into Stage 2 unchanged.

Quick Reference: Message Structure Cheat Sheet

```
# System message — set once, defines agent behavior
{"role": "system", "content": "You are a ..."}

# User message — human input
{"role": "user", "content": "What is 25% of 480?"}

# Assistant message — model text response (final answer)
{"role": "assistant", "content": "The answer is 120."}

# Assistant message — model tool call request
{
    "role": "assistant",
    "content": None,
    "tool_calls": [{
```

```
            "id": "call_abc123",
            "type": "function",
            "function": {
                "name": "calculator",
                "arguments": '{"expression": "480 * 0.25"}'
            }
        }]
}

# Tool result — your code's response to a tool call
{
    "role": "tool",
    "tool_call_id": "call_abc123",  # must match the tool_call id above
    "content": "120.0"
}
```

Memorize this structure. Every other concept in agents is built on it.